SGF v4.0

Documentación Completa del Proyecto

Generado 22/07/2026 - Repo Junmoxia41/SGF - d8eb8e9 fix(theme): cuadro BD en login ahora respeta tema claro/oscuro de la app

1. Introducción

SGF v4.0 (Sistema de Gestión de Facturas ETECSA) es una aplicación cliente/servidor para centralizar la carga, OCR, validación y reporte de facturas telefónicas. Nació como reemplazo de un sistema Oracle legacy con tablas PCELULAR.* y evolucionó a soporte multi-motor (Oracle, SQL Server, SQLite) para funcionar en PCs aisladas sin internet, incluso con antivirus corporativo ESET.

2. Objetivos

- Centralizar facturas en BD enterprise accesible desde varias PCs.
- Permitir carga masiva por PDF con extracción automática (parser + OCR).
- Soporte offline con SQLite como fallback y sincronización PCELULAR.CARGARARCH (solo Oracle).
- Autenticación centralizada con JWT y roles admin/usuario.
- Distribución portable via .zip sin internet (excepto Node.js).

3. Arquitectura General

Cliente (React + Vite) -> HTTP API (Node.js http nativo, sin Express) -> Capa BD multi-motor (database.ts) -> Oracle / SQL Server (mssql) / SQLite (sql.js). El servidor sirve el frontend compilado desde client/dist con fallback SPA.

```
[Browser :3000] -> [Node http server dist/index.js] -> query()/execute()
|-> oraclePool (oracledb) / mssqlPool (mssql) / db (sql.js)
|-> server-data/sgf-data.db
|-> client/dist/index.html
```

4. Stack Tecnológico

Backend

- Node.js 20.x, TypeScript 5.9.3, tsx 4.19.0
- oracledb 6.7.1 (driver Oracle, requiere Instant Client)
- mssql 12.7.0 (wrapper de tedious, puro JS, sin binarios)
- bcryptjs 3.0.3, jsonwebtoken 9.0.2, pdf-parse 1.1.4, sql.js 1.12.0

Frontend

- Vite 8.1.4 (usa rolldown, no esbuild, 0 vulnerabilidades)
- React 18.3+, TailwindCSS 4.1, framer-motion, lucide-react
- pdfjs-dist 4/5 para preview PDF

5. Estructura de Directorios

```
SGF/
```

```
server/src/config, middleware/auth, models/database, routes/*, utils/*
client/src/components/LoginScreen, Header, DbConfigPanel, Tabs
*.bat (instalar-todo, start-servidor, empaquetar-portable)
extraer-node-modules.ps1
```

6. Modelo de Datos

Tres motores con esquemas equivalentes. Tablas SGF_* creadas automáticamente.

Tabla	SQLite	MSSQL/Oracle	Desc
SGF_USUARIOS	ID TEXT PK, USERNAME UNIQUE, PASSWORD_HASH, ROLE, CREATED_AT	ID NVARCHAR(36) PK, USERNAME NVARCHAR(50), PASSWORD_HASH NVARCHAR(255), ROLE NVARCHAR(20), CREATED_AT DATETIME2	Usuarios
SGF_FACTURAS	ID TEXT, CLIENTE, CUENTA, NO_FACTURA, ..., CREATED_AT	ID NVARCHAR(36), CLIENTE NVARCHAR(100), ..., CREATED_AT DATETIME2	Facturas cabecera
SGF_SERVICIOS	ID, FACTURA_ID, NUMERO_SERVICIO	ID, FACTURA_ID...	Lineas servicio
SGF_LOGS	ID, ACCION, NIVEL, CREADO	ID, ACCION, LEVEL, CREATED_AT	Auditoria
SGF_SESIONES	ID, TOKEN UNIQUE, ACTIVO, CREADO, EXPIRA	ID, SESSION_TOKEN NVARCHAR(512) UNIQUE, ACTIVE, CREATED_AT/EXPIRES_AT	Sesiones JWT

Abstracción multi-motor

- nowExpr() -> SYSTIMESTAMP | SYSUTCDATETIME() | datetime('now')
- expireExpr(N), paginationExpr(off,lim), nullableCoalesce
- toMssql / toSqlite traductores, paramNames

7. Endpoints API

Método	Ruta	Auth	Descripción
POST	/api/auth/login	No	Login, retorna JWT, rate limit 30/min
POST	/api/db/test	No	Prueba conexión sin guardar
GET	/api/db/config	No	Config actual y modo
POST	/api/db/connect	No (admin si logueado)	Guarda .env y conecta
POST	/api/db/disconnect	No	Vuelve a SQLite
GET	/api/facturas	Sí	Lista paginada
POST	/api/facturas/upload	Sí	Sube PDF y extrae

8. Instalación

- instalar-todo.bat: detecta Node, proxy, extrae zips con extraer-node-modules.ps1 robusto, npm install fallback, compila tsc + vite.
- start-servidor.bat: arranca server/dist/index.js con auto-recuperación de node_modules.
- empaquetar-portable.bat: genera 3 zips via Compress-Archive.

9. Seguridad

- Passwords bcrypt cost 12, JWT secret en .env obligatorio, 8h expiración.
- Rate limit 30/min 200/15min configurable, TRUST_PROXY para IP real.
- CORS *, isOracleLegacyTablesAvailable solo Oracle toca PCELULAR.*